

LC 3652: Best Time to Buy and Sell Stock Using Strategy

Problem breakdown, mathematical formulation, and sliding window optimization.

prices vector → Stock price on the i^{th} day.

strategy vector → -1 (Buy), 0 (Hold), 1 (Sell).

k → Selects a k -size window to override the strategy.

- First $k/2$ becomes 0 → **Hold**.
- Last $k/2$ becomes 1 → **Sell**.

Output → Maximum possible profit.

1. Mathematical Formulation (Profit Calculation)

Let the original profit without modifications be defined as:

$$\text{Original Profit} = \sum (\text{strategy}[i] \times \text{prices}[i])$$

Gain for First Half

Current Status:

$\text{curr} = \text{strategy}[i] \times \text{prices}[i]$

New Status (Hold):

$\text{new} = 0$

$$\text{gain} = 0 - (S[i] \times P[i])$$

Gain for Second Half

Current Status:

$\text{curr} = \text{strategy}[i] \times \text{prices}[i]$

New Status (Sell):

$\text{new} = \text{prices}[i]$ (You sold and got money)

$$\text{gain} = P[i] - (S[i] \times P[i])$$

2. Understanding the Sliding Window Indices

We use a sliding window of size k to dynamically calculate gains without recalculating the entire sub-array. Let's analyze how indices shift.

Example: $k = 6$, $half = 3$

Initial Window Configuration:

0 1 2 | 3 4 5

For First Half ($k/2$)

Old Indices: 0, 1, 2

New Indices: 1, 2, 3

Remove → $j - k$ → index 0

Add → $j - half$ → index 3

For Second Half ($k/2$)

Old Indices: 3, 4, 5

New Indices: 4, 5, 6

Remove → $j - half$ → index 3

Add → j → index 6

3. C++ Implementation

```
class Solution {
public:
    long long maxProfit(vector<int>& prices, vector<int>& strat, int k) {
        int n = prices.size();
        int half = k / 2;

        long long orig = 0;
        for(int i = 0; i < n; i++) {
            orig += prices[i] * strat[i];
        }

        long long lgain = 0;
        long long rgain = 0;

        for(int i = 0; i < half; i++) {
            lgain += -prices[i] * strat[i];
        }

        for(int i = half; i < k; i++) {
            rgain += prices[i] - prices[i] * strat[i];
        }

        long long maxugain = lgain + rgain;

        for(int j = k; j < n; j++) {
            lgain -= -prices[j - k] * strat[j - k];
            lgain += -prices[j - half] * strat[j - half];

            rgain -= prices[j - half] - (prices[j - half] * strat[j - half]);
            rgain += prices[j] - (prices[j] * strat[j]);

            maxugain = max(maxugain, lgain + rgain);
        }
        return orig + max(maxugain, 0LL);
    }
};
```